DEMYSTIFYING WEB3: BUILDING A TODO APP WITH SOLIDITY AND JAVASCRIPT

PART 2: CONNECTING TO A DEPLOYED SMART CONTRACT USING JAVASCRIPT FRAMEWORKS (REACT-PWA AND WEB3.JS).

AUTHOR: **ABDULHAKEEM MUHAMMED IBIYEMI**
(BLOCKCHAIN/CRYPTOCURRENCY/SMART-CONTRACT DEVELOPER, SOFTWARE ENGINEER, DATA ENGINEER, AI/ML ENGINEER)

PROJECT REPO LINK: https://github.com/ennas-de/TodoApp-Web3
PROJECT LIVE LINK: https://todoapp-web3.netlify.app/
ARTICLE LIVE LINK:
https://docs.google.com/document/d/1diZl4aWi_AqXWwuQ3NJwjn3GOMwL3Zy4

TABLE OF CONTENTS:

1. INTRODUCTIONS

We have successfully deployed our smart contract, our TodoApp smart contract that was written in Solidity and JavaScript programming languages.

After the successful compilation and deployment, we verified the smart contract through Etherscan and Sourcify on the CLI using Hardhat. We also verified the smart contract through the Browser, and we inspected the source codes for the smart contract. These steps are really important as they allow other developers to have trust in our smart contract, and be sure that the contract does not have any malicious code in it. Other developers can also audit our contract as needed.

After the verification process, we tested the smart contract on the CLI again using Hardhat and Ethers.js by writing an `interact.js` script to connect to and call methods in the smart contract. We performed some functions like calling the `getAllTodos()` method and `createTodo(todoDescription)` methods.

With all these, we are sure that our TodoApp smart contract is functioning as expected, and it is time for us to open it up to the world for our clients to have access to it, and leverage it to create amazing tasks.

Let’s get started!

2. CREATE A REACT PROGRESSIVE WEB APP

2.1 Create a new React project using Create React App

STEPS:

I. Search for `create-react-app react pwa` on your browser, and click the link to the Create-React-App documentation for the PWA set up at `https://create-react-app.dev/docs/making-a-progressive-web-app/`.

II. Follow the instructions on the Create React App page for creating a React Progressive Web Application.

a) Run the command below:

$ **npx create-react-app client --template cra-template-pwa**

*** I am using JavaScript version, not TypeScript. Your choice to make.**

b) You'll be asked to `install create-react-app` if you don't have it installed already. Type 'y' and press Enter.

c) The installation process will begin, and a React app will be bootstrapped for you. (Heads up, CRA task a lot longer than Vite, so be patient please)

d) Change directory to the newly created `**client**` directory

$ **cd client**

2.2 Complete the PWA Setup

Register your `service-worker`. This step is needed to make the web app a true PWA, by allowing the app to be available to users even when there is little to no network connection.

e) Open the `src/index.js` file
f) Go to the line with this code: `serviceWorkerRegistration.unregister();`
g) Change the code to: `serviceWorkerRegistration.register();`

III. You can customize the manifest.json file (located inside the `public` dir) if you want to create a more personalized web app that has custom assets like icons, ico, etc. But we would not be doing that in this guide as the focus is on connecting our UI to a smart contract.

2.3 Install all the needed dependencies (MUI and others)

Lets install a React template (MUI) for our pages. I love using templates as the save me a lot of development time for my front end projects.

I. Search for `mui installation for react`. Follow the link to the MUI installation documentation page at `https://mui.com/material-ui/getting-started/installation/`.

II. Follow all the steps in order (use your desired package manager), I am using NPM.

a) You have 2 options to install the MUI components' styling engine:
   i. Using Emotion
   ii. Using Styled-component
   We will be using the Emotion (default, built-in and compatible MUI styling) for this guide.
   I am also using this styling because of its simplicity and flexibility
b) Run the first command: $ **npm install @mui/material @emotion/react @emotion/styled**.
   This installs the MUI core components (using Emotion styling engine) in our project
c) If you ran the command above, you don't need to run the command after (npm install @mui/material @mui/styled-engine-sc styled-components).

III. Install you desired font, but I will be using the Robot font (default, and amazing)
$ **npm install @fontsource/roboto**

IV. Copy the font's imports and paste them in your index.js file

```
import '@fontsource/roboto/300.css';
import '@fontsource/roboto/400.css';
import '@fontsource/roboto/500.css';
import '@fontsource/roboto/700.css';
```

I also removed all the default contents of index.css file so they won't interfere with our styling later.
You can customize these font sizes by importing your own custom settings.

* Note, you can use your own desired font(s) and weights. All are customizable, IT'S REACT and MUI!

Here is what my index.js file looks like now:

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';
import * as serviceWorkerRegistration from
'./serviceWorkerRegistration';
import reportWebVitals from './reportWebVitals';

import '@fontsource/roboto/300.css';
import '@fontsource/roboto/400.css';
import '@fontsource/roboto/500.css';
import '@fontsource/roboto/700.css';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

// If you want your app to work offline and load faster, you can
change
// unregister() to register() below. Note this comes with some
pitfalls.
// Learn more about service workers: https://cra.link/PWA
serviceWorkerRegistration.register();

// If you want to start measuring performance in your app, pass a
function
// to log results (for example: reportWebVitals(console.log))
```

```
// or send to an analytics endpoint. Learn more:
https://bit.ly/CRA-vitals
reportWebVitals();
```

V. Install the MUI Icons:
$ **npm install @mui/icons-material**

With these steps, we are done with our UI setup (for now).
* We will be installing more modules/libraries/packages as needed.

VI. Install React router dom, Web3.js
$ **npm i react-router-dom web3**

3. SETTING UP PAGES FOR CLIENT

Here is the file-system structure for we would be using for this projects:

client/
- **-public/**
- **-node_modules/**
- **-src/**
  - **-assets/**
  - **-components/**
  - **-pages/**
  - **-smart-contract/**
  - **-widgets/**

3.1 Set up the App.jsx file for routing

I. Rename the default `App.js` file to `App.jsx`
II. Import the needed dependencies to create a react-router-dom router element
III. Create a ‘router` object and pass in some template routes to use.
IV. Your initial App.jsx file content should look like this now:

```
import React from "react";
import { createBrowserRouter, RouterProvider } from "react-
router-dom";

import './App.css';

export default function App() {
```

```
  const router = createBrowserRouter([
    {
      path: "/",
      // element: <Landing />,
    },
    {
      path: "/dashboard",
      // element: <Dashboard/>,
    },
  ]);

  return <RouterProvider router={router} />;
}
```

* The path: “/” is the route for our landing page (to be created soon)
* The path: “/dashboard” would be the dashboard route for users to connect with the Smart contract.
  This Dashboard route will load the Dashboard element or page (to be created later).

* Note: If you have a long list of routes, you can extract the `router` part and abstract it into its own component.

3.2 Create Frontend Landing Page

Because we are lazy, and we don’t really have much time re-inventing wheels, we will be leveraging the MUI packages extensively. We don’t need to rebuild pages that has been built for us.

I. Search for `mui landing page template free` :D
II. There are many MUI free and paid templates (that you can choose from).
We will be using this for our project: `https://mui.com/store/items/onepirate/`
* Click on the `Download` button, and it’ll take you to the GitHub page for the project.

III. On the GitHub page, there are a number of files and one folder (modules). We will only be using the contents (files) that we need from all these, as we don’t need to import all (THIS IS MUI!).
For the Landing page, we need ALL the contents of the Landing page of our template.

IV. Right-click and open the `Home.js` file in another tab

V. Right click and open the `modules` directory in another tab

VI. We will copy the contents of the `Home.js` file and paste it into a new file in our project directory

a) Create a new component folder in our project directory inside the `**pages**` folder called **Home**.
b) Create a new file inside the Home directory called `**index.jsx**`
c) Go back to the GitHub repo Home.js tab we opened, and copy all the contents of the Home.js file

VII. From the contents inside the index.jsx file of the Home directory, we can see that we need a lot of contents from the `modules` directory.

VIII. We will create a new directory inside our `components` directory called `Home` and create another folder inside the Home called `modules`. W

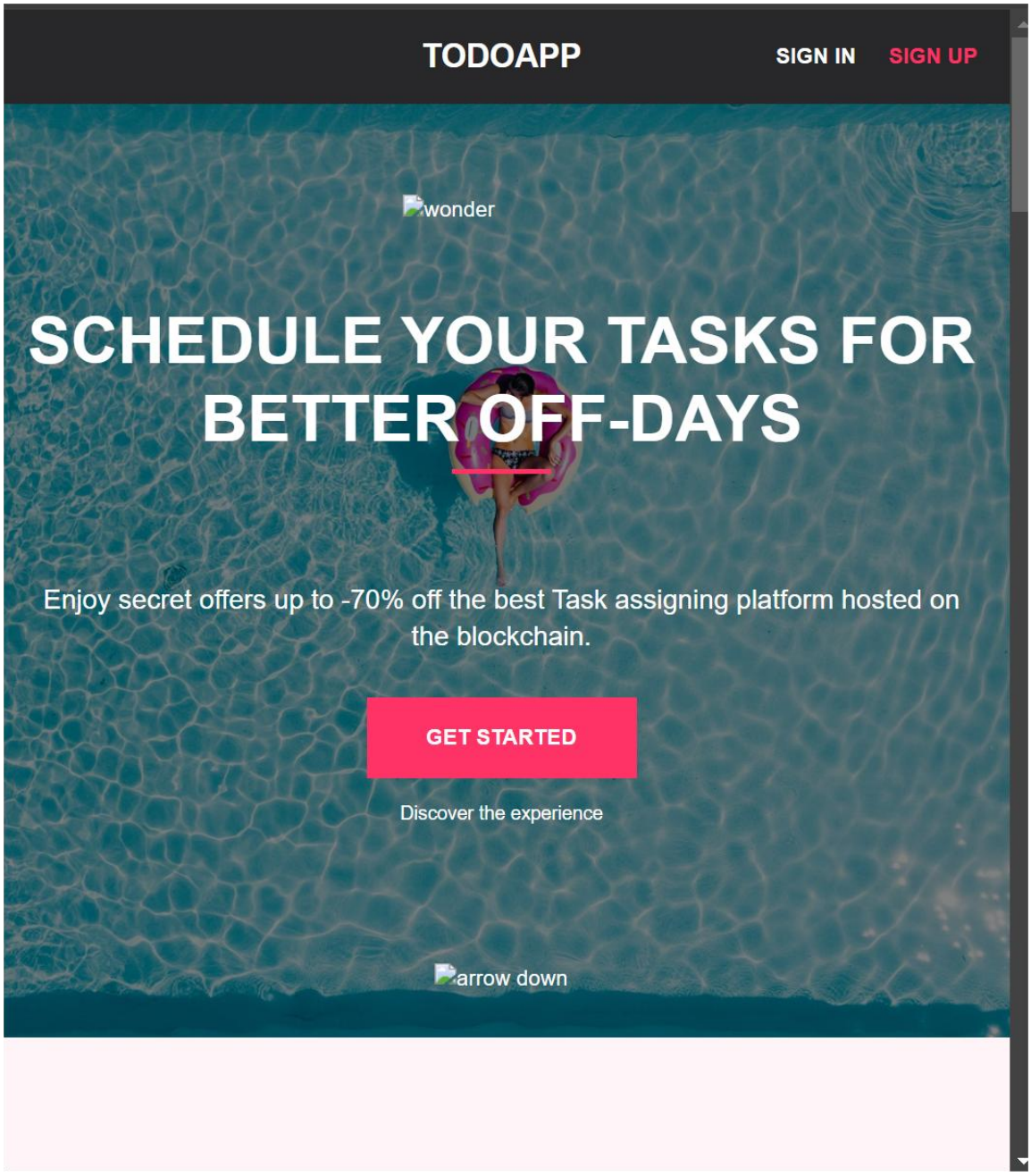
IX. We will create all the files needed from the modules directory and copy their contents into their corresponding files in our project's **component/Home/modules** directory.
* But I am a lazy dev, so what I will do is that I will download the entire repo (MUI repo) containing the project that I need. Then I will open the onepirate directory, copy the Home.js and modules directory.
(To me that's less stressful, and avoids future bugs, WHICH IS VERY IMPORTANT!)

* UPDATE: I rearranged the files and folders in the project to meet my desired structure. The updates would be found in the Client directory of the project (linked above).

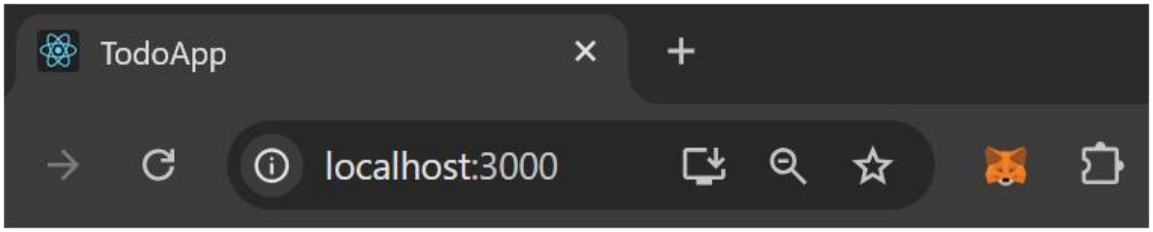
This new structure is to give possibility for future scaling.

* Off Points:
** I moved ALL my projects to WSL2, and it feels amazing.
** Its super fast (way faster than the windows filesystem.
** I can stop worrying about using the windows Bash/Shell or terminal going forward.
** I have always preferred the Linux shell more.
** Though for a windows OS devices, I'll still need to use the terminal (Powershell) once in a while

The new Landing page of our TodoApp Project:

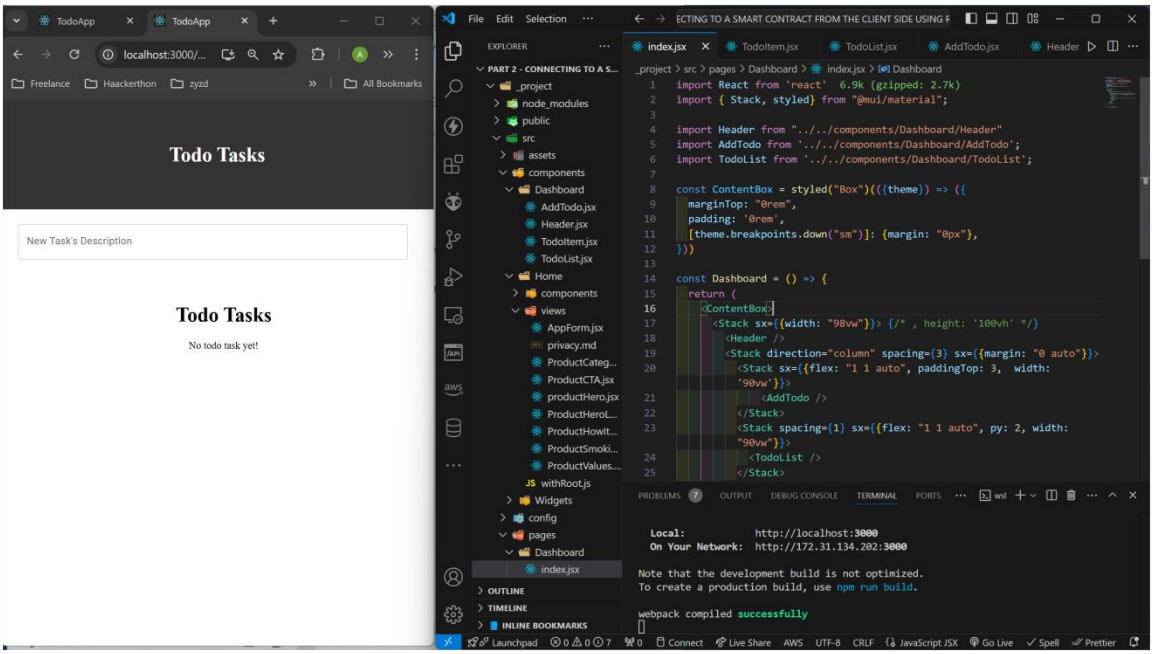


If you look at the search bar, you'll see a download icon (towards the right), Users can click this icons and have the TodoApp installed on their devices (it is a cross-platform application).



3.3 Create the Dashboard page for Interacting with the TodoApp Smart contract
For the Dashboard page, I don't have a template, but I'll search for one. If there's a template that I like, I'll import it into my project (similar to how I imported the Home/Landing template). Else, I will design a basic dashboard with the necessary fields and buttons.
Please find your desired template or you can use the project client directory (from the project repo that is stated at the beginning of the tutorial).

Here is my dashboard for interacting with the TodoApp Smart Contract:



I went ahead to build all the necessary components needed for the project so that we can go ahead and continue with the purpose of the tutorial `**Connecting a React-PWA frontend to a deployed Smart Contract**`.
If there are errors or need for updates later, we will make adjustments.

3.4 **Create other Interaction Components and Pages**
We will now create all other important components we need to interact with our deployed smart contract.

Here are the list of files and or components we need to connect the client UI to the Smart Contract:
I. Blockchain.js (used to make API calls to the smart contract)
II. Interact.jsx (needed to connect the web UI to an Ethereum virtual wallet, like Metamask)
III. Contract-abi.json (this is the contract ABI of the deployed contract, from first part of the series)

A. Interact.jsx
This file as stated earlier is used to enable connection with an Ethereum virtual wallet (we will be using Metamask).

I. ConnectWallet function:
This function is used to check if a virtual wallet is installed on the browser. If yes, the function returns the first address (address currently in use by the user), else, if returns a message that the user should install a wallet extension.

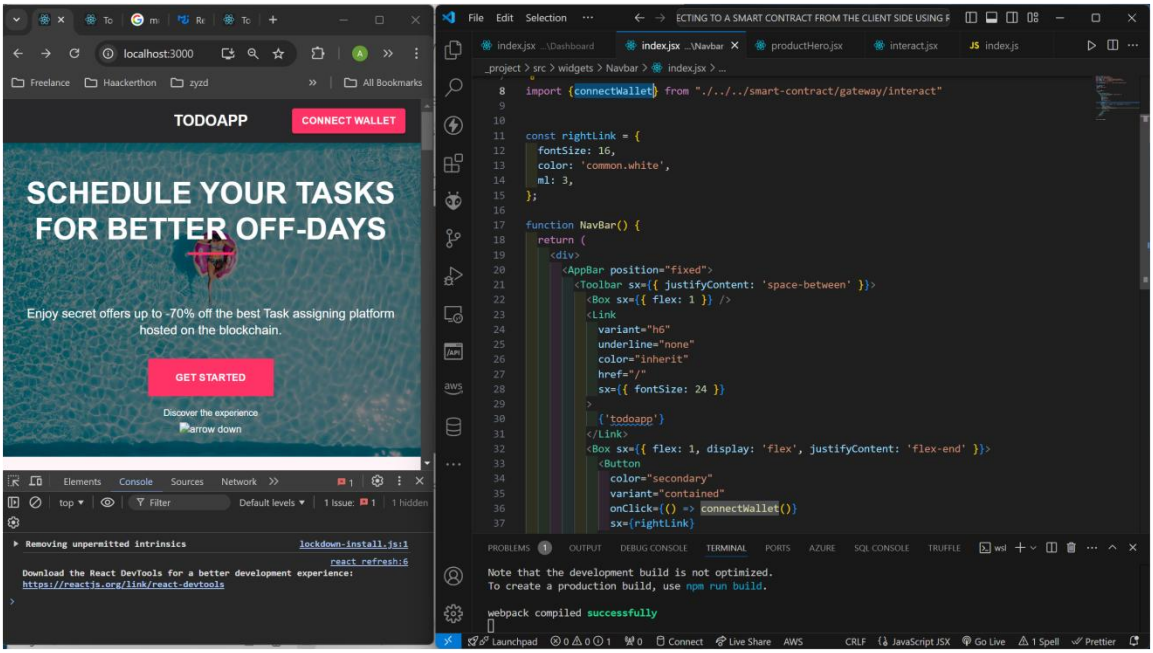
```
export const connectWallet = async () => {
    //check if the user's browser has an ethereum wallet
installed
    if (window.ethereum) {
        try {
            // get all the list of wallet addresses from the
wallet
            const addresses = await window.ethereum.request({
                method: "eth_requestAccounts"
            })

            // return the first address from the list
            return addresses[0];
        } catch (error) {
            console.log({error})
            return false // the user can cancel our the
connection request
        }
    } else { // if the user does not have a wallet installed
        return {
            address: null,
            status: (
                <span>
```

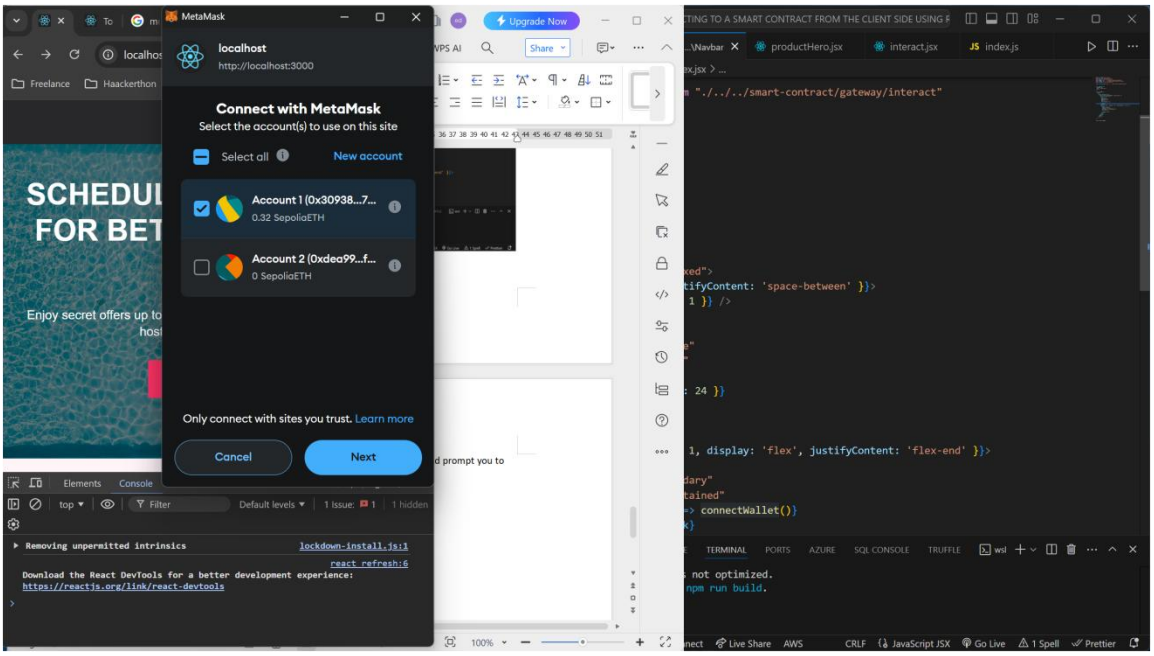
```
                    <p>
                        <a
href="https://metamask.io/download.html">You must install an
Ethereum wallet (e.g Metamask) in your Browser.</a>
                    </p>
                </span>
            )
        }
    }
}
```

Lets import this function into our Landing page (Home.jsx). This will allow the users to click a button and connect their wallet.

I imported the `connectWallet` function into the Navbar component (I have changed the Navbar contents).



When you click on the ‘CONNECT WALLET` button, it will open the Metamask wallet and prompt you to connect a wallet.



Select your preferred wallet address (if there's more than 1), and click Next.

B. Contract-ABI.json

For the contract-abi.json file, we need to import our deployed smart-contract's ABI from the smart-contract directory (first project).

To get the contract ABI, follow these steps below:

I. Open the smart-contract folder (or anything you named your smart contract project directory.

II. Open the `Artifacts` sub-directory. Then open the `contracts` sub-directory.

III. In the contracts directory, you'll find the todo.sol (or what ever name you gave your smart-contract file, it will end with `.sol`).

IV. If you open the todo.sol directory, you'll find the smart contract json file. This file will be named by the given to the smart contract itself (not the filename as mentioned in step III, but the smart contract name itself). The extension of the file will be a `.json`. In our case, we named our smart contract as TodoApp.

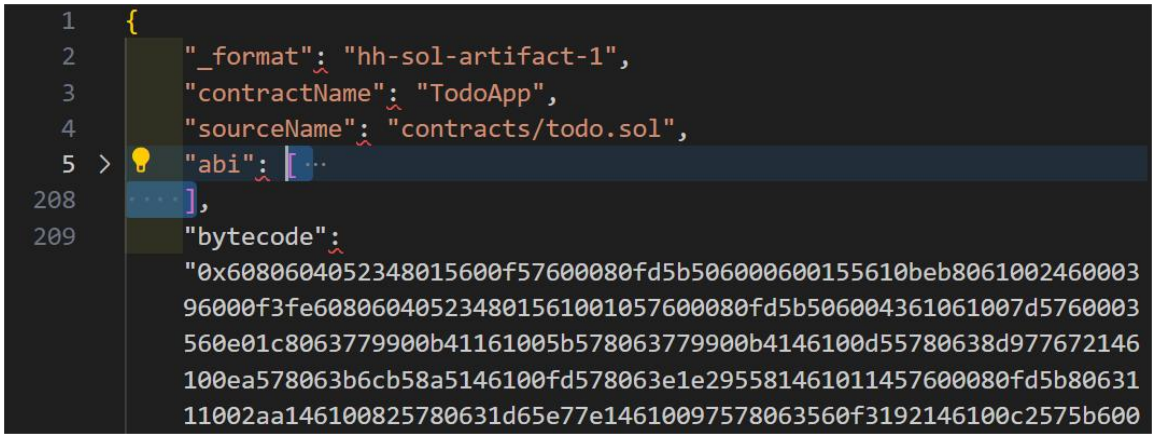
```
contract TodoApp {
    // structure to define a Task
    // this holds a single task object
    struct Task {
        uint256 id;
        string taskDescription;
        bool isCompleted;
    }
```

```
    // Mapping of task id to the Task structure
```

This was our smart contract. As seen above, the name was TodoApp.
Hence, the file we are looking for would be `**TodoApp.json**`.

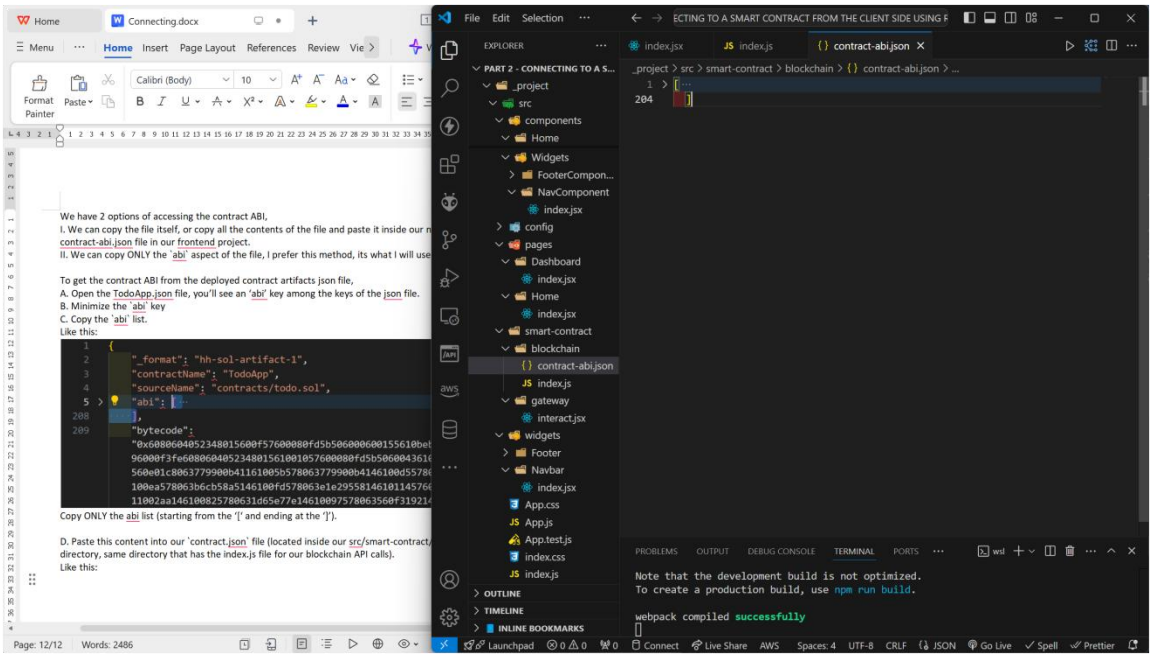
This file contains our contract ABI.
We have 2 options of accessing the contract ABI,
I. We can copy the file itself, or copy all the contents of the file and paste it inside our newly created contract-abi.json file in our Frontend project.
II. We can copy ONLY the `abi` aspect of the file, I prefer this method, its what I will use.

To get the contract ABI from the deployed contract artifacts json file,
A. Open the TodoApp.json file, you'll see an 'abi' key among the keys of the json file.
B. Minimize the `abi` key
C. Copy the `abi` list.
Like this:



Copy ONLY the abi list (starting from the '[' and ending at the ']').

D. Paste this content into our `contract.json` file (located inside our src/smart-contract/blockchain directory, same directory that has the index.js file for our blockchain API calls).
Like this:



As we can see on the opened file `contract-abi.json` file, we have pasted the contract abi from the deployed contract artifacts' json file.

C. Blockchain.js

Lastly, we will build our `blockchain API` file. This file will be used to interact and make API calls to our deployed smart contract.

To write the codes to interact with the smart contract, we need to know the functions available (defined) in our smart contract, we also need to know what parameters each function accepts.

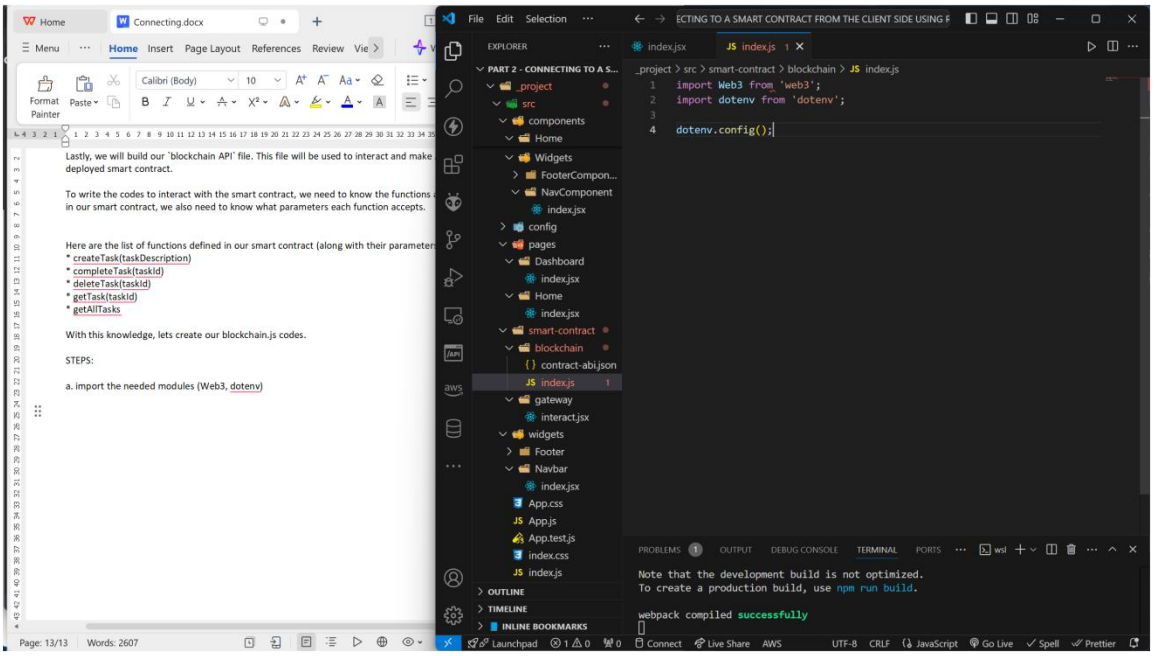
Here are the list of functions defined in our smart contract (along with their parameters):

* createTask(taskDescription)
* completeTask(taskId)
* deleteTask(taskId)
* getTask(taskId)
* getAllTasks

With this knowledge, lets create our blockchain.js codes.

STEPS:

a. import the needed modules (Web3, dotenv)



b. We will import our contract abi (from the contract-abi.json file).

```
import contractABI from './contract-abi.json';
```

c. We will define and import our contract address variable from our .env file.
* Create a .env file and add the contract address of the deployed smart contract to it.

```
CONTRACT_ADDRESS=0x063C80E7D35004feA780fA52cEDa4986d6FC95d3
# TodoApp deployed to: 0x063C80E7D35004feA780fA52cEDa4986d6FC95d3
```

Now we can use it in our blockchain API file (blockchain/index.js).

```
const CONTRACT_ADDRESS = process.env.CONTRACT_ADDRESS;
```

d. Let's create a Web3 object,

```
const web3 = new Web3(window.ethereum);
```

We passed in 'window.ethereum' as a parameter to the Web3 class. This enables Web3 to use the connected Ethereum wallet as the provider for the connection.

e. With all the above steps done, we can now successfully create to our smart contract variable (to be used to make calls to the smart contract. We will create a `contract` variable from web3 object/class created earlier.

```
const contract = new web3.eth.Contract(contractABI,
CONTRACT_ADDRESS);
```

On this line, we added the `contractABI` and the `CONTRACT_ADDRESS` as parameters into the web3 call.

f. Now that we have successfully created our contract variable, we can go ahead and define functions to interact with the smart contract (we can also check out the contract by consoling it to see what methods are available on it).

Lets create our functions

```
// createTask(taskDescription)
export const createTodo = async (taskDescription) => {
    try {
        const account = await getConnectedWallet(); //get
currently connected wallet
        const gasFee = await
contract.methods.createTask(taskDescription).estimateGas({
            from: account
        }) // we need to calculate a gas fee to use for the call
since the function will change the state of the blockchain, hence,
we will be required to pay gas fee

        // we will call the `createTask` method of the smart
contract.
        // we are using the method `SEND` here as this method is
expected to change the blockchain state
        await contract.methods.createTask(taskDescription).send({
            from: account,
            gas: gasFee
        })

        console.log(`New Task created successfully!`)
        return true
    } catch (error) {
        console.log(error);
        return false
    }
}

//  completeTask(taskId)
export const completeTodo = async (taskId) => {
    try {
        const account = await getConnectedWallet();
        const gasFee = await
contract.methods.completeTask(taskId).estimateGas({
            from: account
        })
```

```
        await contract.methods.completeTask(taskId).send({
            from: account,
            gas: gasFee
        })

        console.log(`New Task created successfully!`)
        return true
    } catch (error) {
        console.log(error);
        return false
    }
}

// deleteTask(taskId)
export const deleteTodo = async (taskId) => {
    try {
        const account = await getConnectedWallet();
        const gasFee = await
contract.methods.deleteTask(taskId).estimateGas({
            from: account
        })
        await contract.methods.deleteTask(taskId).send({
            from: account,
            gas: gasFee
        })

        console.log(`Task deleted successfully!`)
        return true
    } catch (error) {
        console.log(error);
        return false
    }
}

// getTask(taskId)
export const getTodo = async (taskId) => {
    try {
        // we will call the `getTask` method of the smart
contract.
        // we are using the method `CALL` here as this method is
not expected to change the blockchain state, hence, we are not to
pay gas fee.
        const todo = await contract.methods.getTask(taskId).call()

        return todo
```

```
    } catch (error) {
        console.log(error);
        return false
    }
}

// getAllTasks
export const getAllTodos = async () => {
    try {
        const todos = await contract.methods.getAllTasks().call()

        return todos
    } catch (error) {
        console.log(error);
        return false
    }
}
```

4. CONNECT TO THE TODOAPP SMART CONTRACT USING WEB3.JS

We have successfully created all the files and codes to connect to the blockchain. All that is left is to connect these functions to the Dashboard.

To make it easy, we will list out all the methods in our smart contract and map them to their corresponding fcuntion API call in the client's blockchain file:
I. createTask -> createTodo(taskDescription)
II. completeTask -> completeTodo(taskId)
III. deleteTask -> deleteTodo(taskId)
IV. getTask -> getTodo(taskId)
V. getAllTasks -> getAllTodos()

Nice, now we know what functions we have in our blockchain/index.js file, and with this information, we know what functions to import into our dashboard and how to call each method.

We will import all these methods into our the individual files that need them (you can use any state management library or framework to handle all the state related logic):

A. Dashboard/index.jsx -> getTodo (not used), getAllTodos:

```
import React, { useEffect, useState } from 'react'
import { Stack, styled} from "@mui/material";
import {getTodo, getAllTodos } from "./../../smart-
contract/blockchain";

import Header from "../../components/Dashboard/Header"
import AddTodo from '../../components/Dashboard/AddTodo';
import TodoList from '../../components/Dashboard/TodoList';

const ContentBox = styled("Box")(({theme}) => ({
  marginTop: "0rem",
  padding: '0rem',
  [theme.breakpoints.down("sm")]: {margin: "0px"},
}))

const Dashboard = () => {
  const [todos, setTodos] = useState([]);

  useEffect(() => {
    const fetchAllTodos = async () => {
      try {
        const resp = await getAllTodos();
```

```
        if (resp) {
          // console.log({resp});
          setTodos(resp)
        }
      } catch (error) {
        console.log({error});
      }
  }

    fetchAllTodos();
  }, [])

  return (
    <ContentBox>
      <Stack sx={{width: "98vw"}}> {/* , height: '100vh' */}
        <Header />
        <Stack direction="column" spacing={3} sx={{margin: "0
auto"}}>
          <Stack sx={{flex: "1 1 auto", paddingTop: 3,  width:
'90vw'}}>
              <AddTodo />
          </Stack>
          <Stack spacing={1} sx={{flex: "1 1 auto", py: 2, width:
"90vw"}}>
            <TodoList todos={todos}/>
          </Stack>
        </Stack>
      </Stack>
    </ContentBox>
  )
}

export default Dashboard
```

B. AddTodo.jsx -> createTodo:

```
import React, { useState } from 'react'
import { FormControl,TextField} from '@mui/material';
import {createTodo } from "./../../smart-contract/blockchain";


const AddTodo = () => {
  const [todoDescription, setTodoDescription] = useState('')

    const handleSubmit = async (e) => {
        try {
            await createTodo(todoDescription);
```

```
            alert("Todo task Created");
        } catch (error) {
            console.log({error});
            alert("Error creating new Todo.")
        }
  }

  return (
    <form onSubmit={handleSubmit}>
        <FormControl fullWidth >
            <TextField
                size='big'
                type='text'
                label="New Task's Description"
                value={todoDescription}
                onChange={(e) =>
setTodoDescription(e.target.value)}
                sx={{mb: 1, width: "100%"}}
            />
        </FormControl>
    </form>
  )
}

export default AddTodo
```

C. TodoItem.jsx -> completeTodo, deleteTodo:

```
import React from 'react'
import {TableCell, TableRow, IconButton, Checkbox} from
'@mui/material';
import DeleteIcon from '@mui/icons-material/Delete';
import {completeTodo, deleteTodo } from "./../../smart-
contract/blockchain";


const TodoItem = ({todo}) => {
    const markAsCompleted = async (todoId) => {
        try {
            await completeTodo(todoId);
            alert("Task marked as completed.")
        } catch (error) {
            alert("Failed to mark task as completed.")
        }
  }
```

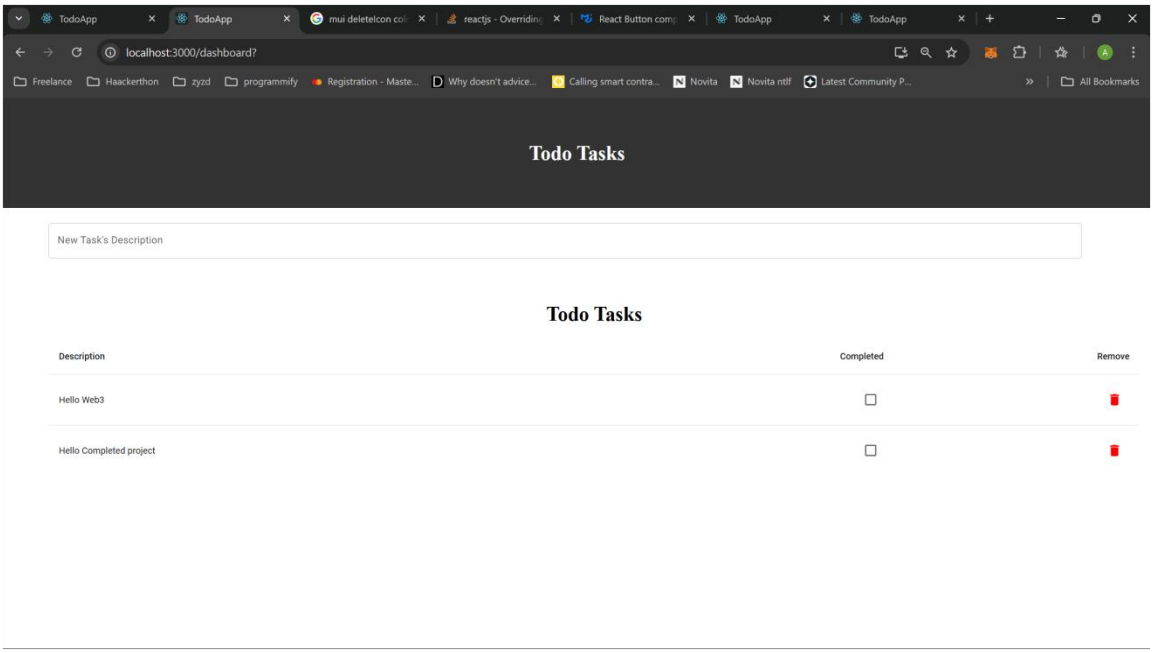
```
    const removeTodo = async (todoId) => {
        try {
            await deleteTodo(todoId)
            alert("Task deleted successfully.")
        } catch (error) {
            alert("Failed to delete task.")
        }
  }

  return (
    <TableRow key={todo.id} sx={{ '&:last-child td, &:last-child
th': {border: 0}}}>
        <TableCell  component="th"
scope="row">{todo.taskDescription}</TableCell>
        <TableCell align="right">
        <   Checkbox onClick={() => markAsCompleted(todo.id)} />
        </TableCell>
        <TableCell align="right">
            <IconButton arial-label="delete" size="large">
                <DeleteIcon onClick={() => removeTodo(todo.id)}/>
            </IconButton>
        </TableCell>
    </TableRow>
  )
}

export default TodoItem
```

5. TEST CONNECTIONS BY INTERACTING WITH THE SMART CONTRACT

A new task was created from the UI input field (press enter to submit).



Other functionalities can be tested and extended as desired.

6. CONCLUSION

6.1 Summary and Conclusion

Web3 has become an amazing and important part of the WWW. Decentralized Applications use cases have grown so much in the last couple of years, and it is projected that it will rise more in the years to come.
Learning to leverage Web3 would be a huge plus for any web developer, because there are huge opportunities for Web3 dApps out there. Also the pay range for blockchain developers are more than the traditional Web2 developers.

It is my hope that this guide will help everyone aiming to venture into Web3 conquer the turbulent tides beginners face (I went through a lot of challenges too).

PWA is also becoming a very important aspect of Frontend development, as a lot of big/giant companies are leveraging the technology. Owning that skill too is a plus for every Frontend/Fullstack developer.

6.2 Whats Next: Unit Testing a Smart Contract (Part 3)

Unit testing software development concepts and functionalities is the new vogue, and to be sincere, a very essential skill to own as a software engineer/developer.
We should have followed the test-driven development approach, but, unit testing is another concept on its own that needs a lot of explaining.
We would be looking at Unit testing our smart contract codes in a later part.

See you then!

Best regards and wishes as we conquer the Web3 Space!
Abdulhakeem Muhammed
(Blockchain/Cryptocurrency/Smart-Contract Developer, Software Engineer (full stack), Data Engineer, AI/ML Engineer)